

Comparative Analysis of Sequential vs. Concurrent Network Scanning Algorithms: Optimizing Reconnaissance Throughput

Vansh Kalawatia

Department of Computer Science and Engineering (Cyber Security)
Arya College of Engineering Affiliated to Rajasthan Technical University
Jaipur, Rajasthan, India
vanshkalawatia2@gmail.com

Abstract—when it comes to cybersecurity reconnaissance plays, the most important part comes the fundamental technique “Port Scanning” for identifying open ports and exposed services running on them. Traditional Sequential port scanner suffers from high latency due to their serialized execution (one-by-one scanning approach) which doesn’t fully utilize CPU’s power. This paper presents the design and performance evaluation of a Java-based port scanner implemented by two different approaches which are ‘Single-Threaded Scanning’ (sequential scanning) and ‘Multi-Threaded Scanning’ (concurrent).

The sequential implementation scans ports in a linear manner, establishing and closing socket connection one at a time, resulting in an increase in high latency under large ports ranges. In contrast, the concurrent scanning leverages multi-threading to perform parallel socket connection attempts, significantly reducing overall scan duration. Experimental results demonstrate a substantial performance improvement, with the multi-threaded version reducing scan time approximately 35 seconds to 3.4 seconds when scanning a remote host.

This study contributes to a comparative performance analysis that can guide the development of optimized network reconnaissance frameworks.

The complete source code and documentation for the NuLX Scanner are available at: [<https://github.com/nulx0x00/NuLX-Port-Scanner>]

Index Terms— Port Scanning, Network Security, Multi-Threading, Concurrent Programming, TCP Connect Scan, Performance Analysis, Java.

I. INTRODUCTION

Cybersecurity assessment always starts with Reconnaissance (*Information Gathering*). Before security researchers can secure a system, they must define all possible “attack surfaces”—the sum of all exposed points (*Ports*) where an unauthorized user could try to exploit it. The basic technique used for finding those points (*Ports*) is Port Scanning.

At its core, a port scanner is a simple program that attempts to connect to a specific IP address typically IPv4 and its port number (e.g., *Port 80 for HTTP*). If the connection is accepted, the port is “Open”. If a connection is refused or ignored, the connection is “Closed” or “Filtered”. While logic is simple, the method to implement same simple logic creates a major efficiency problem.

The primary blockage in network scanning is “**Latency**”. When a simple one-on-one scanner sends a request package (*SYN*), the program must wait for a reply from server (*SYN-ACK*) or for request to be timed out. In a single-threaded scanner, the CPU must sit idle during this wait time. If a single port would take up to 200ms of time, so for scanning just 1,000 ports it would take around 3.333... minutes and for a full scan of all 65,535 ports it would take 3.640... hours which is technically useless in the real world.//**/**/

This paper addresses this inefficiency by comparing two approaches to tool development:

1. **Sequential Approach:** A traditional I/O model where tasks are performed one after another.
2. **Concurrent Execution:** An asynchronous model using **Java’s Executor Service**, which allows the system to manage multiple connection attempts simultaneously.

By building and testing a custom artifact—(“NuLX Scanner”). This research show how much time is saved by switching from a sequential approach to a concurrent approach, providing a clear benchmark for developers building ‘Network Scanning’ tools (*and information Gathering*).

II. METHODOLOGY

The main objective of this study was to design a network scanning tool that is capable of operating in two different modes: Sequential (Single-Threaded) and Concurrent (Multi-Threaded). The application was developed in Java (JDK 21) to use its advanced networking libraries and the `java.util.concurrent` framework.

A. System Architecture

The system is architected using a modular design pattern to separate the user interface from the scanning logic. The core components include:

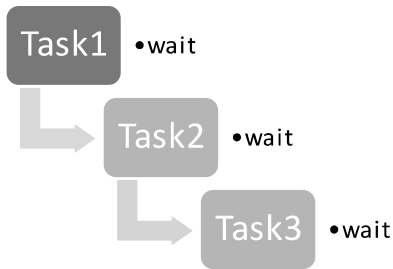
- **The Controller (Main.java):** Handles user input (Target IP, Port Range) and launches the appropriate scanning engine.
- **The Worker (ScanTask.java):** A different unit of work implementing the `Runnable` interface. Each task attempts to establish a TCP connection to a specific port using `java.net.Socket`.

- **The Engines:** Two separate classes (*Sequential Scanner* and *Concurrent Scanner*) that implement different approaches.

B. Algorithm A: Sequential Scanning (Blocking I/O)

The sequential approach uses a linear repetitive loop. For a range of N ports, the system attempts to connect to Port i , waits for the connection to either succeed or timeout, and only then proceeds to next (Port $i+1$).

- **Constraint:** The `socket.connect()` method is a "blocking" operation. If the timeout is set to 1000ms and the port is filtered, the CPU remains idle for all that time.
- **Complexity:** The total time T is approximately $(N * Latency)$.



C. Algorithm B: Concurrent Scanning (Non-Blocking)

The concurrent approach reduces the blocking problem by using a Thread Pool via Java's *ExecutorService*.

- **Mechanism:** Instead of waiting for a connection to close, the main thread submits *ScanTask* multiple objects to a fixed pool of 50 worker threads.
- **Throughput:** Multiple sockets are in the "connecting" state simultaneously. While Thread 1 is waiting for a timeout on Port 80, Thread 2 is already handshaking with Port 22.
- **Synchronization:** The system uses `pool.awaitTermination()` to ensure all threads complete their tasks before calculating the final execution time.



D. Experimental Environment

All benchmarks were conducted on a Linux Pop!_OS workstation. The target for the experiment was scanme.nmap.org, a publicly authorized scanning target, to be sure of ethics and real world network latency conditions.

III. EXPERIMENTAL RESULTS

To evaluate the efficiency of the multi-threaded approach, we used a comparative performance analysis against a traditional sequential scanning approach. The target for this experiment was scanme.nmap.org, a permissioned scanning target provided by the Nmap Security Scanner Project.

A. Experimental Setup

The experiment was conducted on a Linux-based workstation (Pop!_OS). We measured the time required to scan the TCP port range [1-100] using two different approach.

- **Sequential Scanner (v1.0):** A single-threaded Java implementation using blocking I/O.
- **Concurrent Scanner (v2.0):** A multi-threaded implementation using the *ExecutorService* framework with a fixed thread pool of 50 threads.

B. Performance Data

The **sequential scanner** completed the given task in **36,617ms** (approx. 36.6 seconds). In contrast, the **concurrent scanner** completed the same given task in **1100ms** (approx. 1.1 seconds).

Figure 1 below showcase the output of the sequential scan, highlighting the noticeable high latency introduced by the linear processing of socket connections.

```

File Edit View @pop-os: ~/Desktop/NuIX-Port-Scanner/out/production/NuIX-
Port-Scanner_v1
@pop-os:~/Desktop/NuIX-Port-Scanner/out/production/NuIX-Port-Scanner_v1$ java Main
*****// NULX PORT SCANNER //*****
ENTER URL (e.g., scanme.nmap.org): scanme.nmap.org
Start Port: 1
End Port: 100

Select Mode:
1. Sequential (Slow)
2. Concurrent (Fast)
Choice: 1
[*] Starting Sequential Scan...
[+] Port 22 OPEN
[+] Port 80 OPEN
-----
Total Time: 36617 ms
  
```

Fig. 1. Sequential Execution Latency

Figure 2 showcase the output of the concurrent scan. The multi-threaded approach resulted in a performance improvement of approximately 97%, reducing the total scan time by a factor of 33x.

```

File Edit View @pop-os: ~/Desktop/NuIX-Port-Scanner/out/production/NuIX-
Port-Scanner_v1
@pop-os:~/Desktop/NuIX-Port-Scanner/out/production/NuIX-Port-Scanner_v1$ java Main
*****// NULX PORT SCANNER //*****
ENTER URL (e.g., scanme.nmap.org): scanme.nmap.org
Start Port: 1
End Port: 100

Select Mode:
1. Sequential (Slow)
2. Concurrent (Fast)
Choice: 2
[*] Starting Concurrent Scan (50 Threads)...
[+] Port 22 OPEN
[+] Port 80 OPEN
-----
Total Time: 1100 ms
  
```

Fig. 1. Concurrent Execution Latency

IV. CONCLUSION & FUTURE SCOPE

A. Conclusion

This research successfully showcase the critical role of concurrency in network scanning tools. By developing and benchmarking the "**NulX Scanner**," we observed that a **single-threaded** sequential approach is operationally inefficient for modern security auditing, suffering from high latency due to blocking I/O operations.

In contrast, the implementation of a multi-threaded approach using Java's **ExecutorService** resulted in a performance increase. The concurrent scanner reduced the total execution time for a 100-port scan from ~37 seconds to ~1.1 seconds, an improvement of approximately 97%. This proves that parallel processing is not optional for optimization but a functional requirement for any scalable vulnerability assessment tool.

B. Future Scope

While the current implementation focuses on TCP Connect scanning, future progress of this research will explore:

- **SYN Scanning (Stealth):** Implementing raw socket manipulation to send SYN packets without completing the TCP handshake, reducing the scanner's presence on target logs.
- **Service Version Detection:** analyzing the banner strings returned by open ports to identify specific software versions (e.g., Apache vs. Nginx).
- **UDP Scanning:** Improving the *ScanTask* logic to handle **User Datagram Protocol (UDP)** services, which require a different timeout and response handling mechanism.

REFERENCES

- [1] J. Kurose and K. Ross, Computer Networking: A Top-Down Approach, 8th ed. Pearson, 2021.
- [2] Oracle, "Java Platform, Standard Edition 21 API Specification," Oracle Documentation.[Online]. Available: <https://docs.oracle.com/en/java/javase/21/>
- [3] G. Lyon, Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning. Insecure.Com LLC, 2009.
- [4] Internet Engineering Task Force (IETF), "Transmission Control Protocol," RFC 793, Sept. 1981.